

Emergency Smart Flood Alerting System for Safer Driving in Saudi Arabia

Shatha Alsaidy
shathataalsaidy@gmail.com
King AbdulAziz University

Danah Alghanmi
dana.alghanmi0@gmail.com
King AbdulAziz University

Wassan Almutaani
wasanalmutaani@gmail.com
King AbdulAziz University

Wed Aljahdali
wedaaljahdali@gmail.com
King AbdulAziz University

Arwa Basbrain
example@gmail.com
King AbdulAziz University

Abstract

Flood detection is a critical aspect of disaster management, particularly in regions like Saudi Arabia that experience frequent rainstorms. This study evaluates various approaches and models across three key computer vision tasks: classification, detection, and segmentation, specifically in the domain of flood detection. Traditional techniques like Histogram of Oriented Gradients (HOG) and Scale-Invariant Feature Transform (SIFT) were compared with Convolutional Neural Networks (CNNs) models for classification, showing the effectiveness of both approaches. In detection and segmentation, advanced architectures such as YOLO and custom UNet-based models were assessed, with YOLOv8 achieving the highest performance in terms of precision, recall, and IoU. The results highlight the potential of these diverse techniques to enhance disaster response, improve public safety, and provide innovative real-time flood monitoring solutions. These findings contribute to the development of more robust and scalable systems for mitigating the impacts of flooding.

Keywords: Flood Detection, Disaster Response, Saudi Arabia, Computer Vision, Deep Learning, Classification, Detection, Segmentation, SVM, KNN, ANN, HOG, LBP, SIFT, ORB, CNN (Convolutional Neural Networks), EfficientNet, MobileNet, ResNet50, Inception, YOLO.

1 Introduction

Flooding is a recurring natural disaster that significantly impacts urban and rural areas alike, causing damage to infrastructure, loss of life, and economic setbacks. In Saudi Arabia, where heavy rainstorms often result in flash floods, the absence

of efficient flood detection and monitoring systems exacerbates the situation. The increasing availability of high-resolution imagery and advancements in computer vision presents an opportunity to address these challenges through innovative flood detection systems.

This paper examines various machine learning approaches to flood detection, aiming to identify the most effective solutions for real-time monitoring and response. Classical methods such as HOG and SIFT, once dominant in feature extraction tasks, were compared to modern deep learning-based models like CNNs, transfer learning architectures, and specialized detection & segmentation frameworks, such as YOLOv8. By comparing their performance across classification, detection, and segmentation tasks, this study seeks to uncover the most reliable methods for application in flood-prone regions. The findings emphasize the potential of AI-driven systems to transform disaster management strategies, particularly in areas lacking robust monitoring infrastructure.

2 Problem Statement

Flooding in Saudi Arabia poses a severe threat to public safety, infrastructure, and economic stability, particularly due to the country's frequent rainstorms and limited flood detection systems. Despite the high risk, existing solutions are often reactive rather than preventive, relying on manual reporting or delayed data analysis. The absence of real-time flood monitoring mechanisms limits the ability of authorities to respond effectively, leaving vulnerable communities exposed to significant harm. Moreover, the lack of region-specific datasets and models further hinders the development of localized flood detection systems, highlighting the need for scalable, accurate, and automated solutions tailored to the unique challenges of the region.

3 Objectives

Our project aims to create a system that uses computer vision to detect floods in real-time and send alerts to improve road safety in Saudi Arabia. This system uses image classification, object detection, and segmentation models to check and analyze road conditions such as water level, flooding locations, and flooded humans or cars to alert nearby drivers during bad weather. This use of computer vision helps improve public safety and shows how important it can be in responding to natural disasters and building smarter infrastructure.

4 Literature Review

The advancement of flood detection using computer vision has been significantly influenced by research in both classical and deep learning approaches. This section provides a comprehensive review of relevant studies, focusing on image classification, detection, and segmentation techniques, alongside key methods such as transfer learning, dropout, and feature extraction.

4.1 Classical Approaches in Computer Vision

Classical image analysis techniques like Scale-Invariant Feature Transform (SIFT) and Histogram of Oriented Gradients (HOG) have been foundational in image classification, detection, and segmentation tasks. These methods rely on handcrafted features to extract meaningful patterns from images. For example, SIFT detects key points and computes descriptors invariant to scaling and rotation, while HOG captures gradient orientation distributions for object detection. These approaches excel in controlled environments with minimal variability, making them effective for early computer vision applications [1].

Despite their success, classical techniques face challenges in modern applications. They struggle with scalability and performance when processing large datasets, as they cannot dynamically learn features from data. Additionally, their effectiveness diminishes in complex scenarios involving high variability in lighting, occlusion, and object appearance. These shortcomings have paved the way for data-driven approaches like deep learning, which outperform classical methods by learning hierarchical features directly from raw data. However, classical methods still serve as a baseline and complement deep learning in hybrid approaches for some niche applications [2].

4.2 Deep Learning Approaches in Computer Vision

Research has demonstrated that deep learning techniques significantly enhance flood detection capabilities. According to Krizhevsky et al. (2012), Convolutional Neural Networks (CNNs) are highly effective in learning hierarchical features from image data, enabling robust image classification.[3]

Similarly, Ren et al. (2015) proposed Faster R-CNN, a model capable of real-time object detection using region proposal networks, making it suitable for flood-related object detection tasks.[4]

Ronneberger et al. (2015) introduced U-Net, a model designed for precise image segmentation, which excels at separating flooded and non-flooded regions through pixel-level predictions.[5]

Long et al. (2015) developed Fully Convolutional Networks (FCNs), which perform dense predictions by replacing traditional fully connected layers with convolutional ones, further advancing segmentation tasks in flood detection.[6]

Redmon and Farhadi (2018) mentioned that YOLOv5 balances detection speed and accuracy, making it practical for real-time flood detection systems.[7]

4.3 Advanced Techniques in Computer Vision

Transfer learning, dropout, and feature extraction have emerged as core techniques in advancing flood detection through computer vision. Transfer learning, demonstrated by He et al. (2016), leverages pre-trained models such as ResNet and VGG, enabling efficient model training even with limited datasets. This approach accelerates the learning process by utilizing pre-existing knowledge from large-scale datasets.[8]

Dropout, introduced by Srivastava et al. (2014), acts as a regularization method to reduce overfitting by randomly disabling neurons during training, enhancing the model's ability to generalize to unseen data. Feature extraction, crucial in CNN architectures, enables models to capture hierarchical patterns from raw images, significantly boosting classification and detection performance in flood-related tasks.[9]

These advancements facilitate the deployment of robust and scalable flood segmentation and detection systems.

4.4 Related Work

This subsection reviews related papers to gain a deeper understanding of the role of computer vision in addressing challenges in disaster management and emergency response. The reviewed studies highlight various applications of computer vision techniques, such as object detection, classification, segmentation, and real-time data analysis, to tackle issues like flood monitoring, victim identification, and emergency preparedness. These studies provide valuable insights into the strengths and limitations of current methods and propose innovative solutions to enhance decision-making and improve outcomes in disaster scenarios.

According to a review by Lopez-Fuentes et al. (2018), the field of computer vision has been increasingly applied to emergency management to enhance situational awareness and response during natural and human-made crises. Previous studies explored techniques such as fire and flood detection using classical methods like HOG and LBP, as well as advanced deep learning approaches like CNNs and LSTMs, demonstrating promising results in detecting and understanding emergencies. However, limitations in these studies include dataset constraints, lack of generalizability, environmental variability, and insufficient real-time capabilities. The review addresses these gaps by integrating computer vision methods across diverse emergency types, promoting cross-disciplinary innovation, and emphasizing real-time situational awareness, scalability, and adaptability, thereby contributing to improved emergency response and disaster management technologies[10].

Iqbal et al. (2021) conducted a systematic review to explore the application of computer vision technologies in flood management across prevention, preparedness, response, and recovery phases. Previous approaches utilized technologies like Gauge-Cams for non-intrusive river flow monitoring, satellite imagery for hydrodynamic mapping, and UAVs for high-resolution flood assessments. While these methods demonstrated efficacy in water level detection, debris management, and inundation mapping, they faced challenges including high costs, environmental variability, and limited real-time applicability. To address these gaps, the study proposed a taxonomy linking computer vision capabilities to spe-

cific flood management needs and emphasized the potential of hybrid and crowdsourcing approaches for comprehensive disaster response. This review identifies the lack of generalized datasets and the need for integrated solutions as critical barriers to progress in the field[11].

According to Park et al. (2020), effective flood management requires accurate flood depth estimation, especially in urban areas where transportation is heavily impacted by flooding. Previous studies have utilized data from remote sensing, laser radars, and deep learning methods for flood area detection and segmentation. While these approaches demonstrated success in identifying flooded areas, they lacked the ability to estimate flood depth directly and often required fixed-position cameras or reference data, limiting their applicability in dynamic flood scenarios. To address these gaps, Park et al. proposed a vision-based method using Mask R-CNN for vehicle segmentation and VGGNet for image retrieval. The method calculates flood depth by comparing flooded vehicle images to 3D-rendered vehicle models, achieving an average error of 9.40% of tire height, demonstrating its potential to enhance real-time disaster response and flood management systems[12].

Yanbing Bai et al. (2021) proposed a model for detecting permanent and temporary water in flood disasters using Sentinel-1 SAR and Sentinel-2 imagery fused with a deep learning-based segmentation approach. The study utilized the BASNet architecture and the Sen1Floods11 dataset, which includes labeled images from flood events worldwide. To address the class imbalance between water and non-water pixels, the model employed hybrid loss functions combining focal loss, SSIM loss, and IoU loss. The model achieved high accuracy with an overall accuracy (OA) of 92.81% and a mean Intersection over Union (mIoU) of 52.99% on the test dataset. This research improved the detection of floodwater boundaries and provided a robust and automated framework for mapping floodwaters without relying on pre-flood imagery[13].

Study	Focus	Methods Used	Issues Identified	Main Contribution
Lopez-Fuentes et al. (2018)	General emergency management with computer vision.	Reviewed classical (HOG, LBP) and advanced (CNNs, LSTMs) techniques.	Limited datasets, scalability issues, and no real-time systems.	Suggested combining various computer vision methods to improve scalability and adaptability in emergencies.
Iqbal et al. (2021)	Flood management across all disaster phases.	Gauge-Cams, UAVs, and satellite imagery for monitoring water levels and mapping.	High costs, limited real-time capabilities, and environmental variability.	Proposed a taxonomy and hybrid approaches to link computer vision tools to flood management needs.
Park et al. (2020)	Estimating flood depth in urban areas using vehicle images.	Mask R-CNN for detecting vehicles and VGGNet for comparing images with 3D models.	Existing methods couldn't directly estimate flood depth or operate in real time.	Developed a method to estimate flood depth with 9.40% error, enhancing urban flood management.
Yanbing Bai et al. (2021)	Detecting floodwater using satellite imagery.	BASNet deep learning architecture with hybrid loss functions (focal loss, SSIM, IoU).	Class imbalance between water and non-water pixels.	Improved floodwater boundary mapping with 92.81% accuracy without relying on pre-flood images.

Figure 1: Summary of computer vision studies on flood detection and management.

5 Dataset Analysis

Datasets are essential for developing, training, and evaluating machine learning models. Their quality and structure directly impact model performance in tasks like classification, detection, and segmentation. This section focuses on publicly available flood-related datasets we have used for building our system that detects and manages floods.

5.1 Binary classification Flood/NoFlood

The Binary Classification Flood/No-Flood Dataset, sourced from Kaggle, is designed to train and evaluate models for distinguishing between flooded and non-flooded scenes. It consists of approximately 1,000 images evenly divided into 'Flood' and 'No Flood' classes, ensuring a balanced dataset for training, and testing. Key characteristics of the dataset include a wide variety of image resolutions and diverse environmental conditions, such as different lighting, weather patterns, and geographical settings. This diversity ensures that models trained on the dataset can generalize well to real-world flood detection scenarios.

For classification, images are labeled by class (0 for No Flood and 1 for Flood). However, challenges do exist; despite the overall balance, certain subsets might experience class imbalance, potentially affecting model sensitivity. Additionally, while the total sample size is adequate for classification, ensuring high-quality and consistent labels is crucial, as any inaccuracies can degrade model performance[14].

Visualizations:

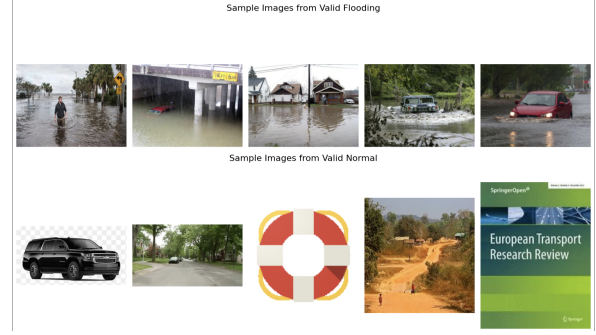


Figure 2: Random Sample of Flood and Non-Flood Images with Labels (0: Non-Flood, 1: Flood).

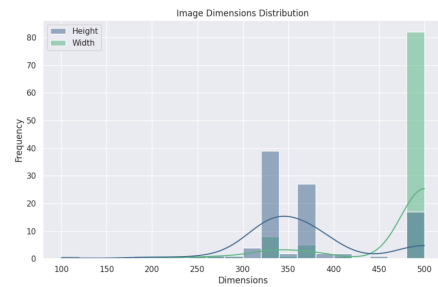


Figure 3: Distribution of Image Dimensions (Height and Width) in the Dataset.



Figure 4: Distribution of Flood (Label: 1) and Non-Flood (Label: 0) Images in the Dataset.

5.2 Flood-Dataset-4gb

The *Flood Dataset (4GB)*, sourced from Kaggle, is specifically designed to support training and testing of machine learning models for binary classification and image recognition tasks related to flooding. The dataset is organized into three subsets: *train*, *test*, and *valid*, ensuring a clear division for model development, validation, and evaluation. Each subset contains images labeled as either "Flooding" or "Normal," making it ideal for binary classification tasks.

The training set includes 2,883 images of flooded scenes and 655 images of normal conditions, providing a substantial dataset for model learning. The validation set contains 42 flooding images and 39 normal images, allowing for fine-tuning and hyperparameter adjustments. The testing set comprises 441 flooding images and 29 normal images, enabling robust evaluation of model performance [15].

Visualizations:

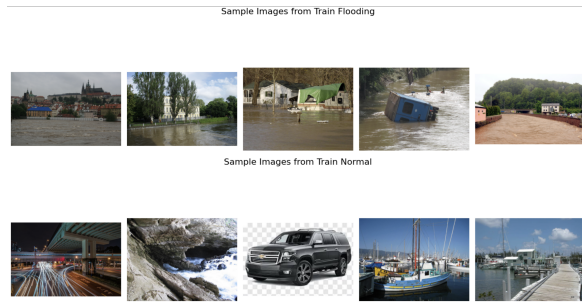


Figure 5: Sample images from the Test dataset: Flooding (top) and Normal (bottom).

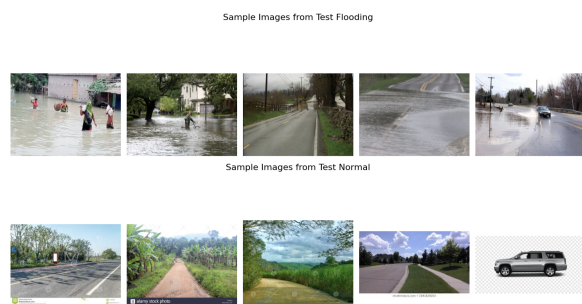


Figure 6: Sample images from the Test dataset: Flooding (top) and Normal (bottom).

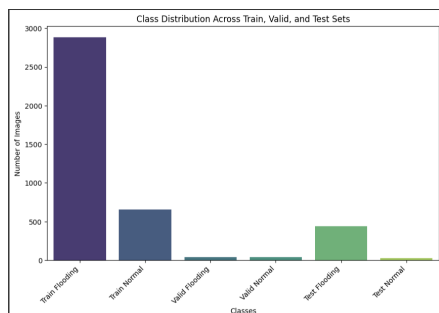


Figure 7: Class distribution of Flooding and Normal images across Train, Validation, and Test sets.

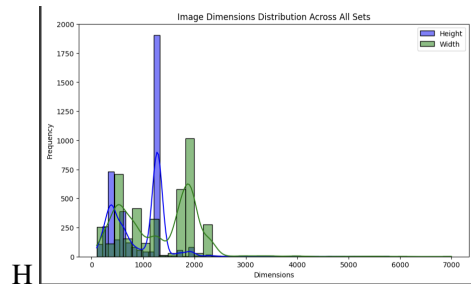


Figure 8: Image Dimensions Distribution Across All Sets (Height and Width).

5.3 Flood Area Segmentation Dataset

The *Flood Area Segmentation Dataset*, sourced from Kaggle, is designed for training and evaluating models to segment water regions in flood-affected areas. It contains **290 images** of flood-hit scenes along with **290 annotated mask images**, all manually labeled using *Label Studio*. The dataset is structured into:

- **Image Folder:** Contains all flood scene images.
- **Mask Folder:** Includes corresponding segmentation masks.
- **Metadata File (`metadata.csv`):** Maps each image to its respective mask for seamless data management.

The total size of the dataset is **581 files**, making it compact yet sufficient for developing segmentation models. The dataset is publicly available under the CC0 license, enabling unrestricted use for research in disaster management, flood monitoring, and emergency planning[16].

Visualizations



Figure 9: Flood image (left) and its flood region mask (right).

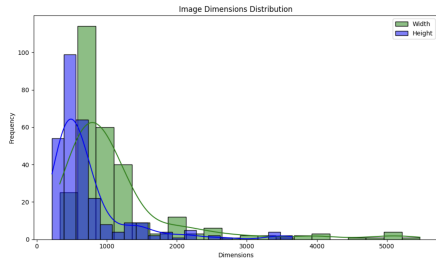


Figure 10: Image Dimensions Distribution Across the Dataset (Height and Width).

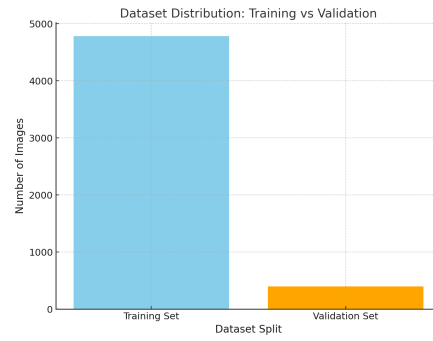


Figure 11: Dataset Distribution (Training Vs Validation).

5.4 Flood Area Dataset

The "flood Area" dataset available on Roboflow is designed for advanced computer vision tasks, specifically segmentation and detection, aimed at identifying and localizing flood-affected areas within various environments. The dataset is split into a training set of 4,782 images (92%) and a validation set of 396 images (8%), with no separate test set provided. It is preprocessed by resizing images to 640x640 pixels and applying auto-orientation, alongside augmentations such as horizontal and vertical flips, cropping (0% minimum zoom and 20% maximum zoom), rotations between -15° and $+15^\circ$, and mosaic transformations. The annotations are provided in two formats: JSON COCO for segmentation tasks, which supports pixel-level annotations, and YOLOv8 for detection tasks, offering bounding box labels for localized flood areas.

The dataset features diverse images captured under various environmental conditions, including different lighting, weather, and geographical settings, making it robust for training models to generalize effectively. However, potential challenges include the absence of a test set for independent evaluation and any inherent class imbalance within subsets, which could affect the model's sensitivity to minority classes. Additionally, the high-resolution and detailed annotations demand substantial computational resources for processing[17].

Visualizations:

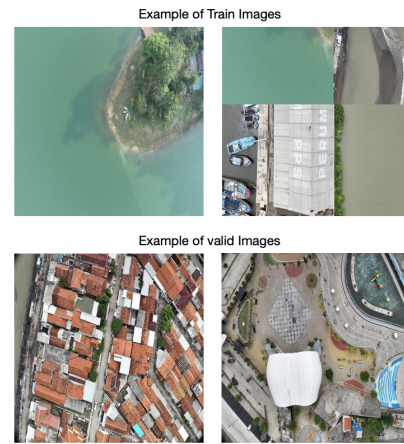


Figure 12: Example Images From Dataset.

5.5 Flooded Objects Dataset

The "Flooded Objects" dataset, sourced from Roboflow, contains 2886 annotated flood images, each of which is 640x640 pixels in size and in grayscale. The dataset includes two detection classes: flooded car and flooded human. It is divided into 2472 images for training, 175 images for validation, and 239 images for testing [18].

Visualizations:



Figure 13: Sample images from the training dataset.



Figure 14: Sample images from the validation dataset.



Figure 18: Sample images from the validation dataset.



Figure 15: Sample images from the test dataset.



Figure 19: Sample images from the test dataset.

5.6 Water Levels Dataset

The 'Water Levels' dataset, sourced from Roboflow, consists of 4,370 annotated color images, each with a resolution of 640x640 pixels, designed for training computer vision-based flood and water level monitoring systems. The dataset includes 13 classes: one for flood detection and 12 representing various water levels. It is divided into 3,825 images for training, 363 for validation, and 182 for testing [19].

Visualizations:

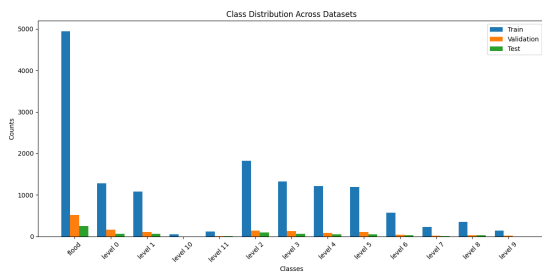


Figure 16: Class distribution of the Water Levels dataset.



Figure 17: Sample images from the training dataset.

6 Preprocessing and Augmentation

In the context of flood detection using computer vision, preprocessing and data augmentation are crucial for ensuring high model accuracy and robust performance. Below are the techniques applied in this study:

6.1 Preprocessing Techniques

Preprocessing transforms raw image data into a suitable format for model training, ensuring consistency and enhancing learning performance. The following preprocessing steps were applied:

- **Image Resizing:** All images were resized to a fixed resolution to maintain uniform input dimensions for the models.
- **Normalization:** Pixel values were scaled to the range $[0, 1]$ to facilitate faster convergence during training.
- **Grayscale Conversion:** For models relying on texture-based features like SIFT and HOG, images were converted to grayscale.

6.2 Data Augmentation Techniques

Data augmentation increases dataset diversity, reducing overfitting and improving model generalization. The following augmentation techniques were implemented:

- **Rotation:** Random rotations up to $\pm 30^\circ$ to simulate different camera angles.
- **Flipping:** Horizontal and vertical flips to address spatial variability in flood scenes.

These preprocessing and augmentation techniques collectively ensured that the models were trained on a diverse and representative dataset, enhancing their ability to detect floods in various real-world conditions.

7 Experimental Setup

The experiments were conducted using Google Colab, using its T4 GPU for accelerated computations. The training environment included Python 3.9 and PyTorch 1.12, with the necessary libraries pre-installed in the Colab environment. The datasets were uploaded to Google Drive to ensure seamless access and integration within the notebook workflow.

In terms of training hyperparameters, they were configured as follows: the learning rate was mostly set to 0.001 with the Adam optimizer, a batch size of 32 was used for the classification models, 16 for the detection models, and lastly 4 for the segmentation models due to memory limits. Training was performed for up to 10 epochs for heavy models and 50 epochs for more efficient ones. Binary cross-entropy loss was used for classification models, bounding box & class prediction loss for the detection models, and lastly, pixel-wise loss was used with the segmentation models.

Evaluation metrics for classical classification models included accuracy, precision, and recall. For CNN models (both custom and pre-trained), the F1-score was additionally used to provide a more balanced assessment. Precision, recall and mean Average Precision (mAP@0.5) are used for the detection models, and intersection over union (IoU) for the segmentation models. Additionally, k-fold cross-validation (k=5) was employed to ensure the results' robustness, where the dataset was partitioned into five subsets, with each subset used as a validation set while training on the remaining four. The final results were obtained by averaging the metrics across all folds, ensuring consistency and reducing variability in performance evaluation.

8 Results and Discussion

In this section, we present and discuss the results of the experiments conducted for the flood detection tasks. The performance of various machine learning models and feature extraction techniques was analyzed for each dataset. Key observations and trends were highlighted, providing in-

sights into the effectiveness of different methods and their potential applications.

8.1 Classification Task

8.1.1 Classical Models

The results of the classical models in Table 1 demonstrate notable variations in model performance depending on the feature extraction method and dataset used. For the Flood dataset, LBP stands out as the most effective feature extraction method, particularly when paired with the SVM model, which achieves the highest cross-validation accuracy 0.7042. ANN also performs well with LBP 0.6804, indicating its adaptability to this feature set. However, the recall values for some combinations, such as SVM with ORB and ANN with ORB, are considerably low, highlighting challenges in correctly identifying positive cases with these methods. Conversely, HOG demonstrates relatively consistent but lower performance across models, while SIFT consistently underperforms, particularly with KNN.

In the Flood/NoFlood dataset, the results improve significantly across all models. SVM with HOG achieves the highest accuracy 0.81, precision, and recall, underscoring its effectiveness with this feature extraction method. KNN and ANN also exhibit strong performance with HOG and LBP, achieving cross-validation accuracies above 0.7. However, SIFT and ORB continue to lag in performance, with ORB producing the lowest accuracies across all models. These results indicate that feature extraction methods like HOG and LBP are particularly well-suited for distinguishing between flood and non-flood conditions, while SIFT and ORB may not capture the critical features necessary for high classification performance.

8.1.2 CNN Models

The results for the CNN models presented in Table 2 indicate strong performance on the Flood dataset, with EfficientNet achieving the highest cross-validation accuracy (0.9848) regardless of whether dropout is applied. MobileNet also performs exceptionally well, achieving a cross-validation accuracy of 0.9870, although its F1-score (0.82) is slightly lower compared to EfficientNet. ResNet50, on the other hand, struggles with this dataset, showing a significantly lower cross-validation accuracy of 0.4321. The custom

CNN model performs moderately, with a cross-validation accuracy of 0.7980, suggesting its potential for improvement.

For the Flood/NoFlood dataset, Inception achieves the best performance, with an impressive cross-validation accuracy of 0.9300, followed by Custom Model 1 (0.9600 with dropout). Custom Model 2 performs well (0.8100 with dropout) but falls behind Inception and Custom Model 1. ResNet50 continues to underperform in this dataset as well, with the lowest cross-validation accuracy of 0.59. These results highlight that pre-trained architectures like EfficientNet, MobileNet, and Inception are highly effective for flood detection tasks, while models like ResNet50 may require further fine-tuning or are less suited for these datasets. Dropout appears to have a marginal impact on performance in most cases, maintaining or slightly reducing accuracy without major performance losses.

8.2 Detection Task

The results for the detection models presented in Table 3 highlight differences in performance across various datasets. YoloV5 achieves moderate training results with a precision of 0.73, recall of 0.77, and MAP50 of 0.76, and maintains reasonable cross-validation metrics (precision of 0.69, recall of 0.67, and MAP50 of 0.69). This suggests that YoloV5 performs consistently but has limitations in generalizing to unseen data.

YoloV8 demonstrates strong cross-validation performance, achieving the highest MAP50 (0.92), precision (0.88), and recall (0.84) among the models. However, its training metrics, particularly recall (0.56) and MAP50 (0.62), are lower compared to YoloV5 and YoloV11. This discrepancy indicates that YoloV8 likely focuses on higher precision during cross-validation but struggles to capture all relevant objects during training.

YoloV11 achieves the best training performance, with the highest MAP50 (0.78) and recall (0.81), demonstrating its ability to detect a wide range of objects effectively during training. Its cross-validation metrics, while slightly lower than YoloV8, remain competitive, with a MAP50 of 0.81 and a balanced precision (0.79) and recall (0.74). This makes YoloV11 a reliable model for scenarios requiring both strong training performance and robust cross-validation results. Overall, the results suggest that YoloV8 is optimal

for high-precision detection tasks, while YoloV11 provides a balanced approach for broader object detection tasks.

8.3 Segmentation Task

The segmentation results presented in Table 4 highlight the performance of various models across different datasets. The Custom Model#1 (Unet-based) demonstrates the weakest performance on the "Water Levels" dataset, with a low IoU of 0.12 and an F1-score of 0.17, suggesting limited segmentation capabilities for this task. On the other hand, Custom Model#2 (Unet-based) performs significantly better on the "Flood Area Segmentation" dataset, achieving a Pixel Accuracy of 0.84, IoU of 0.55, and an F1-score of 0.98, showcasing its suitability for this particular dataset.

The YoloV8 models exhibit superior performance across all datasets. YoloV8#1 achieves strong results on the "Flood Area" dataset, with an IoU of 0.74 and an F1-score of 0.85, demonstrating its effectiveness in handling this dataset. Notably, YoloV8#2 delivers the highest scores overall on the Water Levels dataset, with a Pixel Accuracy of 0.97, IoU of 0.95, and an F1-score of 0.72, emphasizing its robustness and generalization capabilities. These results underscore the effectiveness of YoloV8 models compared to the custom Unet-based models, while also highlighting how dataset characteristics significantly influence model performance. Further analysis will explore the relationship between model architecture and dataset complexity.

9 Conclusion

In conclusion, This study explored machine learning and deep learning models for flood detection across classification, detection, and segmentation tasks, emphasizing the need to align methods with specific objectives. In classification, models like SVM paired with feature extraction techniques such as HOG and LBP delivered reliable results, while CNN architectures like EfficientNet and MobileNet stood out with their impressive accuracy. For detection, YOLOv8 emerged as the top performer with high precision and recall, while YOLOv11 provided a more balanced approach, making it suitable for diverse tasks. In segmentation, YOLOv8 models consistently surpassed Unet-based approaches, achieving higher

IoU and F1-scores across datasets. These results highlight the critical role of dataset properties and model design in optimizing performance, offering valuable insights for developing efficient, real-time flood detection systems.

References

- [1] Ò. Lorente, I. Riera, and A. Rana, *Image classification with classic and deep learning techniques*, 2021. arXiv: 2105.04895 [cs.CV]. [Online]. Available: <https://arxiv.org/abs/2105.04895>.
- [2] V. Bachani, A. Roychowdhury, A. Dasgupta, and H. El-Alfy, "Image segmentation survey: Classical and deep learning methods," in *2024 International Conference on Electrical, Computer and Energy Technologies (ICECET)*, 2024, pp. 1–6. DOI: 10.1109/ICECET61485.2024.10698602.
- [3] A. Krizhevsky, I. Sutskever, and G. E. Hinton, "Imagenet classification with deep convolutional neural networks," *Advances in Neural Information Processing Systems*, vol. 25, pp. 1097–1105, 2012.
- [4] S. Ren, K. He, R. Girshick, and J. Sun, "Faster r-cnn: Towards real-time object detection with region proposal networks," *Advances in Neural Information Processing Systems*, vol. 28, pp. 91–99, 2015.
- [5] O. Ronneberger, P. Fischer, and T. Brox, "U-net: Convolutional networks for biomedical image segmentation," *International Conference on Medical Image Computing and Computer-Assisted Intervention*, pp. 234–241, 2015.
- [6] J. Long, E. Shelhamer, and T. Darrell, "Fully convolutional networks for semantic segmentation," *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, pp. 3431–3440, 2015.
- [7] J. Redmon and A. Farhadi, "Yolov3: An incremental improvement," *arXiv preprint arXiv:1804.02767*, 2018.
- [8] K. He, X. Zhang, S. Ren, and J. Sun, "Deep residual learning for image recognition," *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, pp. 770–778, 2016.
- [9] N. Srivastava, G. Hinton, A. Krizhevsky, I. Sutskever, and R. Salakhutdinov, "Dropout: A simple way to prevent neural networks from overfitting," *Journal of Machine Learning Research*, vol. 15, no. 1, pp. 1929–1958, 2014.
- [10] L. Lopez-Fuentes, J. van de Weijer, M. González-Hidalgo, H. Skinnemoen, and A. D. Bagdanov, "Review on computer vision techniques in emergency situations," *Multimedia Tools and Applications*, vol. 77, pp. 17 069–17 107, 2018.
- [11] U. Iqbal, P. Perez, W. Li, and J. Barthelemy, "How computer vision can facilitate flood management: A systematic review," *International Journal of Disaster Risk Reduction*, vol. 53, p. 102 030, 2021.
- [12] S. Park, F. Baek, J. Sohn, and H. Kim, "Computer vision-based estimation of flood depth in flooded-vehicle images," *Journal of Computing in Civil Engineering*, vol. 35, no. 2, p. 04 020 072, 2021.
- [13] Y. Bai, W. Wu, Z. Yang, *et al.*, "Enhancement of detecting permanent water and temporary water in flood disasters by fusing sentinel-1 and sentinel-2 imagery using deep learning algorithms: Demonstration of sen1floods11 benchmark datasets," *Remote Sensing*, vol. 13, no. 11, p. 2220, 2021.
- [14] K. Pham, *Binary classification: Flood/no flood dataset*, Accessed: [Insert Date Here], 2021. [Online]. Available: https://www.kaggle.com/datasets/khuongpd/binary-classification-floodnoflood?select=devset_images_gt.csv.
- [15] *Flood-dataset-4gb*. [Online]. Available: <https://www.kaggle.com/datasets/phamtuananhk16hl/flood-dataset-4gb/code>.
- [16] F. Karim, *Flood area segmentation dataset*, Accessed: [Insert Date Here], 2021. [Online]. Available: <https://www.kaggle.com/datasets/faizalkarim/flood-area-segmentation>.
- [17] L. N. Hoang, *Flood dataset*, <https://universe.roboflow.com/long-nguyen-hoang-9ecmq/flood-4oelx>, Open Source Dataset, visited on

2024-12-14, Feb. 2024. [Online]. Available:
<https://universe.roboflow.com/long-nguyen-hoang-9ecmq/flood-4oelx>.

- [18] S. T. Dhamad, *Flooded objects dataset*, Accessed: 2024-12-14, 2024. [Online]. Available: <https://universe.roboflow.com/sahar-turki-dhamad/flooded-objects>.

- [19] Q. Ali, *Water level sindh dataset*, Accessed: 2024-12-14, 2024. [Online]. Available: <https://universe.roboflow.com/qurban-ali-qvlhx/water-level-sindh>.

Table 1: Classical Models Classification

Model	Feature Extraction Method	Training			Cross Validation Average Accuracy
		Accuracy	Precision	Recall	
Flood Dataset					
SVM	HOG	0.6265	0.6355	0.6265	0.6449
SVM	SIFT	0.5649	0.5539	0.5649	0.5591
SVM	LBP	0.7217	0.7230	0.2981	0.7042
SVM	ORB	0.6800	0.5775	0.2284	0.6629
KNN	HOG	0.6306	0.6078	0.6306	0.5958
KNN	SIFT	0.5886	0.5583	0.5886	0.5691
KNN	LBP	0.6895	0.5510	0.4819	0.6771
KNN	ORB	0.6068	0.4225	0.1616	0.6149
ANN	HOG	0.7175	0.7121	0.7175	0.6931
ANN	SIFT	0.5782	0.5619	0.5782	0.5823
ANN	LBP	0.6847	0.6517	0.6847	0.6804
ANN	ORB	0.6619	0.5231	0.0947	0.6408
Flood/NoFlood Dataset					
SVM	HOG	0.81	0.81	0.81	0.81
SVM	SIFT	0.73	0.72	0.73	0.71
SVM	LBP	0.64	0.59	0.58	0.6834
SVM	ORB	0.61	0.56	0.63	0.5758
KNN	HOG	0.78	0.74	0.76	0.75
KNN	SIFT	0.74	0.73	0.74	0.73
KNN	LBP	0.64	0.57	0.73	0.6688
KNN	ORB	0.52	0.47	0.50	0.5398
ANN	HOG	0.81	0.81	0.81	0.81
ANN	SIFT	0.71	0.70	0.71	0.7056
ANN	LBP	0.78	0.76	0.64	0.6595
ANN	ORB	0.60	0.54	0.55	0.6249

Table 2: CNN Models Classification

Model	Dropout	Training				Cross Validation Average Accuracy
		Accuracy	Precision	Recall	F1-Score	
Flood Dataset						
EfficientNet	Yes	0.91	0.91	0.91	0.91	0.9840
EfficientNet	No	0.91	0.91	0.91	0.91	0.9848
ResNet50	Yes	0.72	0.71	0.72	0.7204	0.4321
ResNet50	No	0.73	0.73	0.73	0.7323	0.4321
Custom Model	Yes	0.79	0.80	0.79	0.79	0.8420
MobileNet	Yes	0.98	0.82	0.84	0.82	0.9870
Flood/NoFlood Dataset						
Custom Model 1	Yes	0.78	0.74	0.78	0.76	0.9600
Custom Model 1	No	0.75	0.67	0.84	0.75	0.9400
Custom Model 2	Yes	0.80	0.80	0.80	0.80	0.8100
Custom Model 2	No	0.84	0.76	0.69	0.73	0.8300
Inception	No	0.95	0.95	0.95	0.95	0.9300
ResNet50	No	0.60	0.71	0.51	0.59	0.5900

Table 3: Detection Models Results

Model	Dataset	Training			Cross Validation		
		Precision	Recall	MAP50	Precision	Recall	MAP50
YoloV5	Flooded_Objects	0.73	0.77	0.76	0.69	0.67	0.69
YoloV8	Flood_Area	0.73	0.56	0.62	0.88	0.84	0.92
YoloV11	Water_Levels	0.68	0.81	0.78	0.79	0.74	0.81

Table 4: Segmentation Models Results

Model	Dataset	Training					Cross Validation
		Pixel Accuracy	IoU	Precision	Recall	F1-score	IoU
Custom Model#1 (Unet-based)	Water_Levels	0.74	0.12	0.22	0.16	0.17	0.12
Custom Model#2 (Unet-based)	Flood_Area_Segmentation	0.84	0.55	0.98	0.98	0.98	0.73
YoloV8#1	Flood_Area	0.74	0.74	0.99	0.74	0.85	0.80
YoloV8#2	Water_Levels	0.97	0.95	0.81	0.65	0.72	0.88